studeersnel

Báo cáo Mô hình xe 2 bánh tự cân bằng - Nhóm Mỹ Hiền, Tùng Hải, Xuân Nguyên

IT Hua Phung (Ho Chi Minh City University of Technology)

Scan to open on Studeersnel

Studocu is not sponsored or endorsed by any college or university
Downloaded by Du Di?p (diepdu2005@gmail.com)

ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ

----------o0o----------



ĐỒ ÁN MÔN HỌC

THIẾT KẾ VÀ ĐIỀU KHIỂN XE 2 BÁNH TỰ CÂN BẰNG

GVHD: Thầy TRẦN HOÀNG QUÂN
SVTH: Nguyễn Võ Hồng Mỹ Hiền - 2010260
Phạm Tùng Hải – 2013080
Võ Dương Xuân Nguyên - 2010469

TP. HỒ CHÍ MINH, THÁNG 5 NĂM 2023

Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

Downloaded by Du Di?p (diepdu2005@gmail.com)

LỜI CẢM ƠN

Đề tài này được chúng em hoàn thiện trong quá trình học bộ môn Thiết kế hệ thống nhúng dưới sự dẫn dắt của thầy Trần Hoàng Quân trong suốt học kỳ 222. Chúng em xin gửi lời cảm ơn sâu sắc đến thầy, cũng như sự nhiệt tình giảng dạy và hỗ trợ chúng em trong suốt thời gian qua,

Tp. Hồ Chí Minh, ngày 16 tháng 5 năm 2023 .

Sinh viên đại diện



Võ Dương Xuân Nguyên

Dit document is beschikbaar op studeersnel

Downloaded by Du Di?p (diepdu2005@gmail.com)

TÓM TẮT ĐỒ ÁN

Đồ án này trình bày về việc thiết kế xe 2 bánh tự cân bằng trong đó bao gồm các nội dung chủ yếu như sau:

- Đọc giá trị góc nghiêng từ cảm biến MPU6050.
- Cấp tín hiệu điều khiển động cơ STEP từ bộ điều khiển PID.

Downloaded by Du Di?p (diepdu2005@gmail.com)

MỤC LỤC

1. GIỚI THIỆU....3
1.1 Tổng quan....3
1.2 Nhiệm vụ đề tài....3
1.3 Phân chia công việc trong nhóm....3
2. LÝ THUYẾT....4
3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG....6
4. THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM (NẾU CÓ)....7
5. KẾT QUẢ THỰC HIỆN....12
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....13
6.1 Kết luận....13
6.2 Hướng phát triển....13
7. TÀI LIỆU THAM KHẢO....13

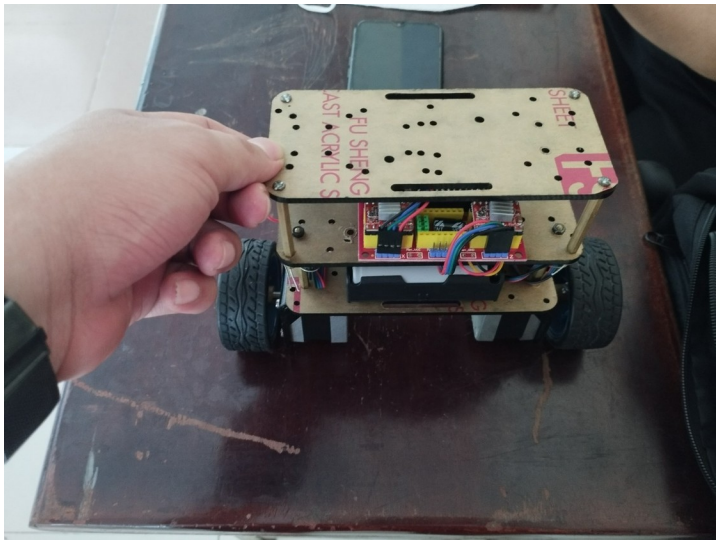
Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

1. GIỚI THIỆU

1.1 Tổng quan

Đề tài xe hai bánh tự cân bằng đặt ra những vấn đề cần nghiên cứu như sau:

- Thiết kế và khử nhiễu để góc nghiêng đọc được từ MPU6050 chính xác nhất.
- Lập trình để truyền tín hiệu điều khiển động cơ STEP.
- Thiết kế bộ diều khiển PID để truyền tín hiệu điều khiển động cơ.



Hình 1. *Mô hình xe 2 bánh tự cân bằng thực tế*

1.2 Nhiệm vụ đề tài

Cụ thể về các nhiệm vụ cần phải giải quyết khi thực hiện đề tài:

Nội dung 1: Tìm hiểu nguyên lý hoạt động của động cơ STEP.

Nội dung 2: Tìm hiểu về cảm biến MPU6050 và thực hiện khử nhiễu.

Nội dung 3: Thiết kế bộ điều khiển PID cho động cơ STEP dựa trên góc nghiêng đọc được từ MPU6050.

1.3 Phân chia công việc trong nhóm

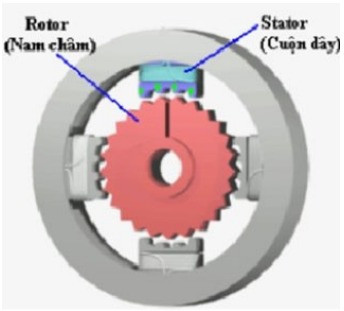
Phân công nhiệm vụ:

Downloaded by Du Di?p (diepdu2005@gmail.com)

- Võ Dương Xuân Nguyên: Thiết kế bộ điều khiển PID và điều khiển động cơ STEP. Yêu cầu thời gian nghiên cứu: 2 tháng.
- Nguyễn Võ Hồng Mỹ Hiền: Đọc và khử nhiễu cảm biến MPU6050. Yêu cầu thời gian nghiên cứu: 2 tháng.
- Phạm Tùng Hải: Thiết kế và lắp phần cứng. Yêu cầu thời gian nghiên cứu: 2 tháng.

2. LÝ THUYẾT

a. Điều khiển động cơ STEP:



Hình 2. *Minh họa động cơ STEP*

- Cấu tạo của động cơ STEP: Rotor và Stator.

  + Rotor: Một dãy các lá nam châm vĩnh cửu xếp chồng lên nhau, trên các lá nam châm lại được chia thành các cặp cực sắp xếp đối xứng với nhau.

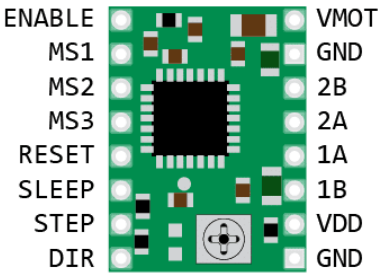
  + Stator: Được cấu tạo bằng sắt từ, chia thành nhiều rãnh nhỏ để đặt cuộn dây.
- Nguyên lí hoạt động của động cơ STEP như sau: Đưa tín hiệu điện vào Stator theo thứ tự và một tần số nhất định. Số lần chuyển mạch sẽ bằng tổng số góc quay của Rotor, chiều quay và tốc độ quay của Rotor phụ thuộc vào thứ tự chuyển đổi và tần số chuyển đổi.



Hình 3. *Động cơ STEP NEMA 17*

Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

- Động cơ STEP NEMA 17: có các chế độ hoạt động 200, 400, 800 và 1600 bước. Trong mô hình xe 2 bánh tự cân bằng, chúng em sử dụng chế độ 1600 bước của động cơ STEP.



Hình 4. *Driver A4988 điều khiển động cơ STEP.*

- Cũng trong mô hình này, chúng em sử dụng Driver A4988 để điều khiển động cơ STEP, chủ yếu sử dụng các chân sau:

  + ENABLE: Cho phép Driver hoạt động cấp tín hiệu cho động cơ STEP.

  + MS1, MS2 và MS3: Chân chọn chế độ bước cho động cơ STEP. (“000” ứng với số bước là 200 và “111” ứng với số bước là 1600)

  + Chân STEP: Chân cấp tín hiệu xung điều khiển động cơ STEP.

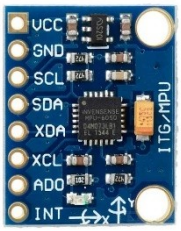
  + DIR: Chân điều hướng quay cho động cơ STEP.

  + VMOT: Chân cấp áp cho động cơ STEP, ở đây cấp nguồn 12V sử dụng pin Lithium.

  + VDD: Chân cấp nguồn cho tín hiệu logic điều khiển Driver A4988.

  + 1A 1B 2A 2B: Chân xuất tín hiệu trực tiếp đến động cơ STEP.

b. Cảm biến MPU6050:



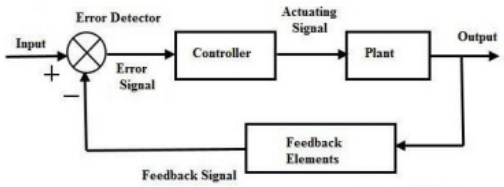
Hình 5. *Cảm biến góc nghiêng MPU6050*

- MPU6050 được sử dụng để đo 6 thông số: 3 trục Góc quay (Gyro) và 3 trục gia tốc hướng (Accelerometer) với độ chính xác lên đến 0.01 độ, kết nối với vi điều khiển thông qua giao tiếp SPI hoặc I2C thông qua 2 chân SDA và SCL (ở đây sử dụng giao tiếp I2C).

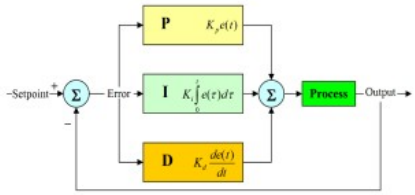
Downloaded by Du Di?p (diepdu2005@gmail.com)

- Địa chỉ phần cứng của MPU6050 tùy vào tín hiệu trên chân AD0, nếu AD0 ở mức HIGH thì MPU có địa chỉ phần cứng là 0x69, nếu AD0 ở mức LOW thì MPU có địa chỉ phần cứng là 0x68 (ở đây đặt địa chỉ MPU mặc định là 0x68).
- Giá trị 3 trục gia tốc hướng được lưu tại địa chỉ 0x43 với tổng cộng 6 bytes (mỗi hướng 2 bytes), tương tự 3 trục góc quay được lưu tại địa chỉ 0x3B với tổng cộng 6 bytes.

c. Bộ điều khiển PID:



Sơ đồ điều khiển hệ thống vòng kín



$$u(t) = K_p \left(e(t) + \frac{1}{T_I} \int_0^t e(t)d\tau + T_D \frac{de(t)}{dt} \right)$$

$$G_{PID}(s) = K_p + K_i \frac{1}{s} + K_d s$$

- Bộ điều khiển PID tính toán giá trị điều khiển ngõ ra với đầu vào là sai số so với giá trị đặt và giá trị điều khiển được tính toán thông qua 3 khâu: Khâu tỉ lệ P (Proportional), khâu tích phân I (Integral) và khâu vi phân D (Derivative).
- Tác động của mỗi khâu đến ngõ ra:

  + Khâu P: Giảm sai số nhanh nhưng nhạy cảm với nhiễu (nhiễu càng lớn thì sẽ bị thay đổi tỉ lệ thuận với hệ số khâu P).

  + Khâu I: Tích lũy sai số với ưu điểm triệt tiêu sai số nhanh, giảm thời gian đáp ứng và tăng thời gian xác lập, nhưng gây ra vọt lố cao.

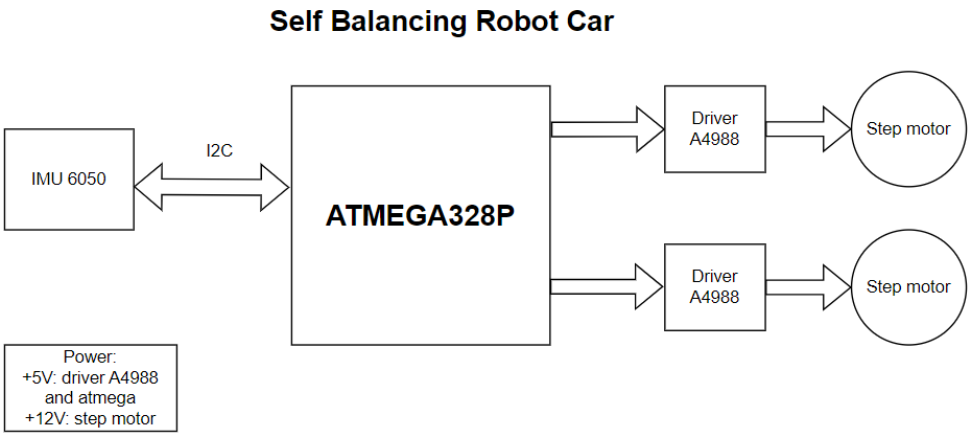
  + Khâu P: Vi phân của sai số với ưu điểm giả sai số xác lập, nhưng bù lại tăng thời gian đáp ứng.

3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG

- **Yêu cầu thiết kế**
  - Đặt MPU6050 ở vị trí đồng trục với bánh xe, thiết kế phải càng thấp càng tốt để dễ giữ vị trí cân bằng.

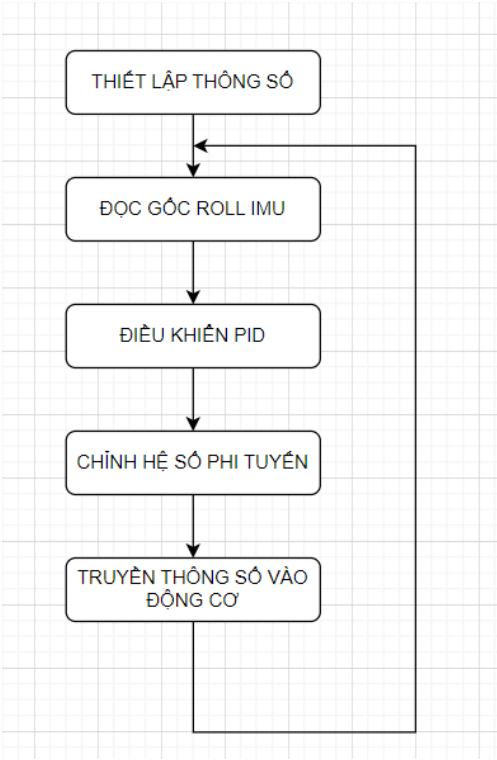
Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

- **Sơ đồ khối mô hình xe 2 bánh tự cân bằng**



4. THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM

- Yêu cầu đặt ra cho phần mềm
  - Đọc và khử nhiễu giá trị đọc về từ MPU6050.
  - Điều chỉnh hệ số PID để đạt kết quả đáp ứng nhanh và vọt lố thấp.
- Lưu đồ giải thuật



Downloaded by Du Di?p (diepdu2005@gmail.com)

- Code thực hiện:
  - Định nghĩa các chân và các giá trị cơ bản như địa chỉ phần cứng MPU và tốc độ baud.

```
#include <avr/io.h>

# define MPU6050_ADDRESS 0x68
# define Enable       8           //D8                //PORTB 0
# define Step_3       7           //D7                //PORTD 7
# define Step_2       6           //D6                //PORTD 6
# define Step_1       5           //D5                //PORTD 5
# define Dir_3        4           //D4                //PORTD 4
# define Dir_2        3           //D3                //PORTD 3
# define Dir_1        2           //D2                //PORTD 2
# define MS3          9           //D9                //PORTB
1 //các chân MS3 cua 2 MOtor1 và MS3 Motor2 nối chung
# define MS2          10          //D10               //PORTB
2 //các chân MS2 cua 2 MOtor1 và MS2 Motor2 nối chung
# define MS1          11          //D11               //PORTB
3 //các chân MS1 cua 2 MOtor1 và MS1 Motor2 nối chung
# define BAUD_RATE 9600
# define UBRR_VALUE ((F_CPU / (BAUD_RATE * 16UL)) - 1)
```

  - Khởi tạo Mode cho các chân.

```
void  pin_INI() {
  DDRB |= (1 << Enable) | (1 << Step_1) | (1 << Step_2) | (1 <<
Step_3) | (1 << Dir_1) | (1 << Dir_2) | (1 << Dir_3);
  DDRC |= (1 << MS1) | (1 << MS2) | (1 << MS3);

  PORTB &= ~((1 << Enable) | (1 << Step_1) | (1 << Step_2) | (1 <<
Step_3) | (1 << Dir_1) | (1 << Dir_2) | (1 << Dir_3));
  PORTC |= (1 << MS1) | (1 << MS2) | (1 << MS3);
}
```

  - Khởi tạo timer 2 với độ chia là 40 và kích hoạt chương trình ngắt.

```
void timer_INI() {
  TCCR2A = 0;
  TCCR2B = 0;
  TCCR2B |= (1 << CS21);
  OCR2A = 39;
  TCCR2A |= (1 << WGM21);
  TIMSK2 |= (1 << OCIE2A);
}
```

  - Khởi tạo UART với tốc độ baud là 9600.

Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

```
void uart_INI() {
  UBRR0H = (uint8_t)(UBRR_VALUE >> 8);
  UBRR0L = (uint8_t)(UBRR_VALUE);
  UCSR0B = (1 << RXEN0) | (1 << TXEN0);
  UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}
```

- Khai báo các biến cơ bản bên trong code.

```
int8_t Dir_M1, Dir_M2, Dir_M3;
volatile int Count_timer1, Count_timer2, Count_timer3;
volatile int32_t Step1, Step2, Step3;
int16_t Count_TOP1, Count_BOT1, Count_TOP2, Count_BOT2, Count_TOP3,
Count_BOT3;
float Input, Output, I, Input_last, M, Motor;
float Filter_AngleX;

float Kp = 25.0;
float Ki = 0.01;
float Kd = 0.01;

float  Offset = -3.3;
unsigned long loop_timer;
```

- Thiết lập chương trình ngắt của TIMER 2 để truyền tín hiệu vào chân STEP và kích tín hiệu chiều quay của động cơ.

```
ISR(TIMER2_COMPA_vect) {
  if (Dir_M1 != 0) {
    Count_timer1++;
    if (Count_timer1 <= Count_TOP1)PORTD |= 0b00100000;
    else PORTD &= 0b11011111;
    if (Count_timer1 > Count_BOT1) {
      Count_timer1 = 0;
      if (Dir_M1 > 0)Step1++;
      else if (Dir_M1 < 0)Step1--;
    }
  }

  if (Dir_M3 != 0) {
    Count_timer3++;
    if (Count_timer3 <= Count_TOP3)PORTD |= 0b10000000;
    else PORTD &= 0b01111111;
    if (Count_timer3 > Count_BOT3) {
      Count_timer3 = 0;
      if (Dir_M3 > 0)Step3++;
      else if (Dir_M3 < 0)Step3--;
    }
  }
}
```

- Xác định chiều quay và số bước của động cơ STEP.

Downloaded by Du Di?p (diepdu2005@gmail.com)

```
void Speed_L(int16_t x) {
  if (x < 0) {
    Dir_M1 = -1;
    PORTD &= 0b11111011;
  }
  else if (x > 0) {
    Dir_M1 = 1;
    PORTD |= 0b00000100;
  }
  else Dir_M1 = 0;

  Count_BOT1 = abs(x);
  Count_TOP1 = Count_BOT1 / 2;
}

void Speed_R(int16_t x) {
  if (x < 0) {
    Dir_M3 = -1;
    PORTD &= 0b11101111;
  }
  else if (x > 0) {
    Dir_M3 = 1;
    PORTD |= 0b00010000;
  }
  else Dir_M3 = 0;

  Count_BOT3 = abs(x);
  Count_TOP3 = Count_BOT3 / 2;
}
```

- Khởi tạo giao tiếp I2C.

```
void I2C_Init() {
  TWBR = ((F_CPU / 100000UL) - 16) / 2;
  TWCR = (1 << TWEN);
}

void I2C_Start() {
  TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
  while (!(TWCR & (1 << TWINT)));
}

void I2C_Write(uint8_t data) {
  TWDR = data;
  TWCR = (1 << TWINT) | (1 << TWEN);
  while (!(TWCR & (1 << TWINT)));
}

uint8_t I2C_ReadACK() {
  TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWEA);
```


Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

```
  while (!(TWCR & (1 << TWINT)));
  return TWDR;
}

uint8_t I2C_ReadNACK() {
  TWCR = (1 << TWINT) | (1 << TWEN);
  while (!(TWCR & (1 << TWINT)));
  return TWDR;
}
```

- Viết chương trình đọc MPU6050.

```
void MPU6050_ReadAccelerometer(float Filter_AngleX) {
  int16_t accelX, accelY, accelZ;
  float weight = 0.1, AngleX;
  I2C_Start();
  I2C_Write((MPU6050_ADDRESS << 1) | 0);
  I2C_Write(0x3B);
  I2C_Start();
  I2C_Write((MPU6050_ADDRESS << 1) | 1);
  accelX = (I2C_ReadACK() << 8) | I2C_ReadACK();
  accelY = (I2C_ReadACK() << 8) | I2C_ReadACK();
  accelZ = (I2C_ReadACK() << 8) | I2C_ReadNACK();
  AngleX = (atan(accelY / sqrt(accelX * accelX + accelZ * accelZ)) *
180 / 3.14);
  Filter_AngleX = weight * AngleX + (1.0 - weight) * Filter_AngleX;
}

int main()
{
  pin_INI();
  timer_INI();
  uart_INI();
  while(1)
  {
```

- Thực hiện điều khiển PID (ở đây xem Setpoint = 0 nên giá trị góc đọc được cũng chính là sai số).

```
  MPU6050_ReadAccelerometer(Filter_AngleX);
  Input = Filter_AngleX + Offset;
  I += Input * Ki;
  if (I < -400) I = -400;
  if (I > 400) I = 400;
  Output = Kp * Input + I + Kd * (Input - Input_last);
  Input_last = Input;

  if (Output > -2.5 && Output < 2.5)Output = 0;
  if (Output >= 400) Output = 400;
  if (Output <= -400) Output = -400;
```

- Hiệu chỉnh hệ số phi tuyến để truyền vào động cơ STEP

Downloaded by Du Di?p (diepdu2005@gmail.com)

```
    if (Output > 0) M = 410 - (1 / (Output + 9)) * 5500;
    else if (Output < 0)  M = -410 - (1 / (Output - 9)) * 5500;
    else M = 0;

    if (M > 0)Motor = 400 - M;
    else if (M < 0)Motor = -400 - M;
    else Motor = 0;

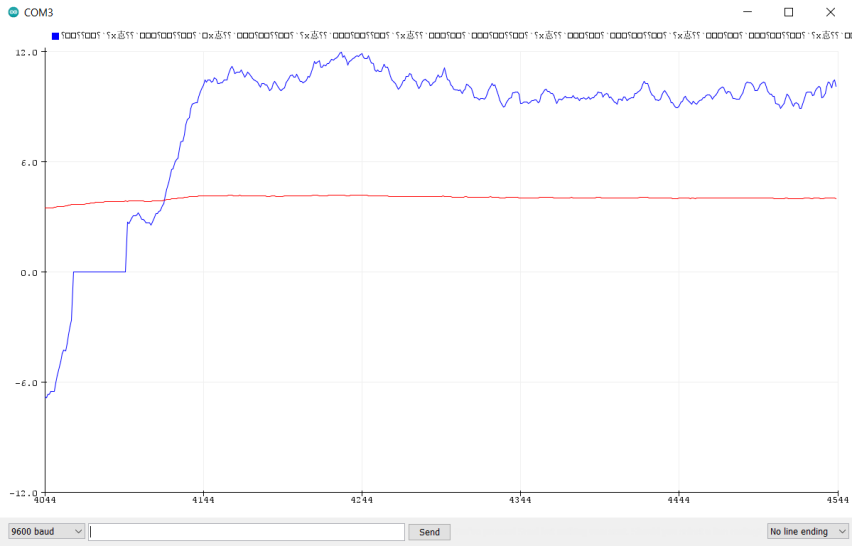
    Speed_L(Motor);
    Speed_R(Motor);

    while (loop_timer > micros());
    loop_timer += 1000;
    }
    return 0;
}
```

5. KẾT QUẢ THỰC HIỆN

Sử dụng chức năng Plot của phần mềm Arduino IDE, ta được:

- Đường màu đỏ là giá trị góc MPU6050 đọc về, đường màu xanh là kết quả đáp ứng đại diện cho tốc độ quay. Có thể thấy, tốc độ động cơ dần tiến về ổn định và nhỏ khi giá trị góc đọc được nhỏ.



Dit document is beschikbaar op studeersnel
Downloaded by Du Di?p (diepdu2005@gmail.com)

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Thông qua quá trình thực hiện đề tài, tuy xe 2 bánh tự cân bằng chưa thể tự đứng vững được do tốc độ hồi tiếp còn chậm, do hệ số PID chưa thích hợp. Dù vậy, đáp ứng động cơ tỏ ra hiệu quả khi tốc độ quay sẽ thay đổi nhanh hay chậm tùy theo góc quay nhiều hay ít. Đồng thời, giá trị góc quay mà MPU6050 đọc về đạt ổn định, không bị dao động nhiều, chứng tỏ việc khử nhiễu đã tỏ ra hiệu quả mong muốn.

6.2 Hướng phát triển

Mô hình xe 2 bánh tự cân bằng hiện tại vẫn chưa đáp ứng được khả năng chịu tải trọng lớn hoặc với mô hình lớn hơn sẽ khó mà tìm được hệ số PID thích hợp. Vì vậy, trong tương lai, định hướng của nhóm sẽ cải tiến bộ điều khiển khác, cụ thể là bộ điều khiển LQR (tương tự như bài toán điều khiển cân bằng con lắc ngược) để thực hiện xe cân bằng, đồng thời sử dụng động cơ và khung xe có thể chịu được tải trọng lớn hơn.

TÀI LIỆU THAM KHẢO

[1] https://components101.com/sensors/mpu6050-module

[2] https://www.researchgate.net/publication/334731253_A_Tutorial_on_Modelling_and_Control_of_Two-_Wheeled_Self-Balancing_Robot_with_Stepper_Motor

Downloaded by Du Di?p (diepdu2005@gmail.com)